

ООП

Контекст выполнения функции — this

this (контекст) — это ссылка на объект, в котором вызывается функция. Смысл тот же, что у местоимения "он" в предложении: вместо повторения имени объекта внутри метода используют this.

```
const user = {
  username: "Victor",
  showName() {
    // ❌ плохо — повторяем имя объекта
    console.log(user.username);
  },
};
```

```
const user = {
  username: "Victor",
  showName() {
    // ✅ хорошо — используем this
    console.log(this.username);
  },
};
```

Главное правило: значение this определяется в момент **вызова** функции, а не в момент её объявления (кроме стрелочных функций).

Глобальный контекст

```
function foo() {
  console.log(this);
}
foo();
```

- Без строгого режима (non-strict mode) — this указывает на объект window (глобальный объект браузера).
- В строгом режиме ("use strict") — this будет undefined.

Контекст метода объекта

Если функцию вызвали как метод объекта — this указывает на этот объект.

```
const user = {
  username: "Poly",
  showThis() {
    console.log(this);
  }
};
user.showThis(); // {username: "Poly", ...}
```

Контекст зависит от способа вызова, а не от места создания функции:

```
"use strict";
function showThis() {
  console.log(this);
}

const user = { username: "Poly" };
user.showContext = showThis;

user.showContext(); // this = объект user
showThis();         // this = undefined (вызов в глобальном контексте)
```

Это одна и та же функция — результат разный, потому что зависит только от способа вызова.

Метод call()

Позволяет вызвать функцию с явно заданным контекстом (this).

```
foo.call(thisArg, arg1, arg2, ...)
```

Параметр	Назначение
thisArg	объект, который станет this внутри функции
arg1, arg2...	аргументы функции, передаются по одному

```
function greet(str) {
  console.log(`${str}, ${this.username}, комната ${this.room}!`);
}

const mango = { username: "Mango", room: 27 };

greet.call(mango, "Welcome"); // "Welcome, Mango, комната 27!"
```

Метод apply()

Работает как call(), но аргументы передаются **массивом**, а не по одному.

```
foo.apply(thisArg, [arg1, arg2, ...])
```

```
greet.apply(mango, ["Welcome"]); // "Welcome, Mango, комната 27!"
```

Разница call vs apply — только в форме передачи аргументов. Во всём остальном методы идентичны.

Метод bind() и потеря контекста

В отличие от call/apply, которые вызывают функцию сразу, bind() не вызывает функцию, а возвращает **новую функцию** с уже закреплённым контекстом — вызвать её можно позже.

```
const boundFoo = foo.bind(thisArg, arg1, arg2, ...)
```

Проблема, которую решает bind — потеря контекста при сохранении метода в переменную:

```
"use strict";
const customer = {
  username: "Jacob",
  sayHello() {
    console.log(`Hello, ${this.username}!`);
  }
};

const greet = customer.sayHello;
greet(); // TypeError — this стал undefined
```

Решение:

```
const greet = customer.sayHello.bind(customer);
greet(); // "Hello, Jacob!"
```

bind() и колбэки

Та же проблема часто возникает, когда метод передают как колбэк-функцию:

```
const customer = {
  firstName: "Jacob",
  lastName: "Mercer",
  getFullName() {
    return `${this.firstName} ${this.lastName}`;
  }
};

function makeMessage(callback) {
  console.log(`Processing an application from ${callback()}`);
}

makeMessage(customer.getFullName); // TypeError — контекст потерян
makeMessage(customer.getFullName.bind(customer)); // ✅ "Processing an application from Jacob Mercer"
```

Стрелочные функции (arrow functions) и this

Ключевое отличие: у стрелочных функций `this` определяется **местом объявления**, а не местом вызова, и никогда не меняется после создания.

```
const showThis = () => console.log(this);
showThis(); // this = window (взято из места объявления)
```

Даже если присвоить стрелку методом объекта — `this` всё равно возьмётся снаружи:

```
const showThis = () => console.log(this);
const user = { username: "Mango" };
user.showContext = showThis;

user.showContext(); // this = window, а не user
```

Стрелочные функции **игнорируют строгий режим** — в глобальном контексте `this` всегда `window`, даже с `"use strict"`.

Полезный кейс — стрелка внутри обычного метода "запоминает" контекст родителя:

```
const hotel = {
  username: "Resort hotel",
  showThis() {
    const foo = () => console.log(this);
    foo(); // this = hotel (взято из showThis, где foo объявлена)
  },
};
hotel.showThis();
```

Методы `call`, `apply`, `bind` **не действуют** на стрелочные функции — контекст изменить нельзя:

```
const showThis = () => console.log(this);
showThis.call({ username: "Mango" }); // всё равно this = window
```

Что важно запомнить про `this` в стрелках:

1. Контекст определяется местом объявления, а не вызова.
2. Строгий режим игнорируется — в глобальном контексте всегда `window`.
3. `call`, `apply`, `bind` не влияют на `this` стрелочных функций.

Алгоритм определения `this`

Универсальный порядок проверки, который позволяет точно определить значение `this` в любой функции.

Шаг	Вопрос	Если Да	Если Нет
1	Это стрелочная функция?	<code>this</code> берётся из внешней (родительской) области видимости	переходим к шагу 2
2	Использованы <code>call</code> , <code>apply</code> или <code>bind</code> ?	<code>this</code> — объект, переданный первым аргументом	переходим к шагу 3
3	Функция вызвана как <code>object.method()</code> ?	<code>this</code> — объект слева от точки	переходим к шагу 4
4	Скрипт выполняется в строгом режиме?	<code>this = undefined</code>	<code>this = window</code>

Проверять нужно именно в этом порядке — например, стрелочная функция (шаг 1) "побеждает" даже вызов через `call/apply/bind`.

Прототипы

Прототип объекта

Встроенные методы массивов (`push`, `map`, `reduce...`) и методы функций (`call`, `bind...`) существуют благодаря механизму **прототипного наследования** (`prototype inheritance`): объекты связываются в цепочку так, что если свойство не найдено в самом объекте, поиск автоматически продолжается в объекте-прототипе.

Связующее звено — служебное свойство `[[Prototype]]` (явно его не меняют, оно работает "под капотом").

Метод `Object.create(obj)` создаёт новый объект и связывает его с `obj` как с прототипом:

```
const animal = { legs: 4 };

const dog = Object.create(animal);
dog.name = "Mango";

console.log(dog.name); // "Mango" — своё свойство
console.log(dog.legs); // 4 — берётся из прототипа animal
```

Прототип — это "резервное хранилище" свойств и методов, которое автоматически используется при поиске.

Проверка прототипа — isPrototypeOf()

```
objA.isPrototypeOf(objB)
```

Проверяет, является ли objA прототипом для objB. Возвращает true/false.

```
const animal = { legs: 4 };
const dog = Object.create(animal);
dog.name = "Mango";

console.log(animal.isPrototypeOf(dog)); // true
console.log(dog.isPrototypeOf(animal)); // false — наследование не работает в обратную сторону
```

Собственные и несобственные свойства

- **Собственное свойство** (own property) — принадлежит самому объекту.
- **Несобственное свойство** — унаследовано от прототипа.

```
const animal = { legs: 4 };
const dog = Object.create(animal);
dog.name = "Mango";

console.log(dog.hasOwnProperty("name")); // true — своё
console.log(dog.hasOwnProperty("legs")); // false — из прототипа
```

obj.hasOwnProperty(key) проверяет, принадлежит ли свойство key именно объекту obj.

Перебор собственных свойств

Цикл for...in перебирает **и свои, и унаследованные** свойства — это часто мешает:

```
for (const key in dog) {
  console.log(key); // "name", "legs" — попало и унаследованное
}
```

Решение — фильтр через hasOwnProperty:

```
for (const key in dog) {
  if (dog.hasOwnProperty(key)) {
    console.log(key); // "name"
  }
}
```

Проще и чаще используют на практике Object.keys(obj) / Object.values(obj) — они и так возвращают только свои ключи/значения, без унаследованных:

```
console.log(Object.keys(dog)); // ["name"]
console.log(Object.values(dog)); // ["Mango"]

for (const key of Object.keys(dog)) {
  console.log(key); // "name"
}
```

Цепочки прототипов

Прототип сам может иметь свой прототип — так строится цепочка. Строится она справа налево (с конца):

```
const objC = { c: "objC prop" };

const objB = Object.create(objC);
objB.b = "objB prop";

const objA = Object.create(objB);
objA.a = "objA prop";
```

Получается цепочка: `objA` → `objB` → `objC`. Благодаря ей `objA` имеет доступ к свойствам `objB` и `objC`:

```
console.log(objA.a); // "objA prop" — своё
console.log(objA.b); // "objB prop" — из objB
console.log(objA.c); // "objC prop" — из objC
console.log(objA.x); // undefined — нет нигде в цепочке
```

Алгоритм поиска свойства: интерпретатор ищет свойство в самом объекте → если не находит, переходит по `[[Prototype]]` к прототипу → и так до конца цепочки → если нигде не найдено, возвращается `undefined`.

Конец цепочки прототипов

В конце любой цепочки прототипов всегда находится ссылка на базовый класс `Object` — от него происходит тип данных объекта.

Чем отличается хороший код от плохого

Раньше критерий был прост: работает без ошибок — хорошо, есть ошибки — плохо. Но в реальных проектах (кодовая база от 100 000 строк) важны и другие критерии:

1. **Надёжность** — стабильная работа даже у большого и сложного ПО (программное обеспечение).
2. **Масштабируемость** — лёгкая адаптация под растущую нагрузку.
3. **Адаптивность** — гибкость, возможность быстро менять функционал/интерфейс.
4. **Стоимость** — низкие затраты на разработку повышают конкурентоспособность.

Чтобы писать такой код, нужен либо огромный опыт, либо знание правильных парадигм программирования.

Процедурное программирование

Программа строится как набор функций, выполняющих действия над данными.

- **Процедуры (функции)** — основная единица программы, принимают аргументы, обрабатывают их, возвращают результат.
- **Локальные и глобальные переменные** — локальные доступны только внутри функции, глобальные — во всей программе.

```
const baseSalary = 30000;
const overtime = 10;
const rate = 20;

const getWage = (baseSalary, overtime, rate) => {
  return baseSalary + overtime * rate;
};

getWage(baseSalary, overtime, rate);
```

Подход простой, но с ростом сложности программы связь между данными и функциями их обработки теряется — становится неудобно.

Объектно-ориентированное программирование (ООП)

ООП — парадигма, в которой программа строится как набор объектов. Каждый объект представляет сущность (пользователь, магазин, автомобиль) и объединяет данные (свойства) и методы работы с ними.

Тот же пример в стиле ООП:

```
const employee = {
  baseSalary: 30000,
  overtime: 10,
  rate: 20,
  getWage() {
    return this.baseSalary + this.overtime * this.rate;
  },
};

employee.getWage();
```

Здесь почти нет глобальных переменных — методы используют свойства самого объекта. Это упрощает структуру, разработку и поддержку сложных программ.

Класс

Класс — шаблон (чертёж), который описывает структуру и поведение будущих объектов: какие у них будут свойства и методы.

Аналогия: проектируя автомобиль, мы заранее описываем все детали (двигатель, колёса) и их взаимодействие. Это и есть класс — сам по себе не автомобиль, а чертёж для его производства.

Экземпляр класса

Экземпляр (объект, инстанция) — конкретный представитель класса, созданный по этому "чертежу". У каждого экземпляра свои значения свойств, но одинаковая структура и поведение.

Класс — абстрактный автомобиль на чертеже, экземпляр — конкретная машина во дворе.

Интерфейс класса

Интерфейс — набор свойств и методов класса, доступных для работы с экземпляром (аналог приборной панели автомобиля).

Важен баланс: простой интерфейс легко использовать, но он менее гибкий; гибкий интерфейс мощнее, но сложнее и легче ошибиться.

Классы: объявление

```
class User {  
  // Тело класса  
}
```

Создание экземпляра — через оператор `new`:

```
class User {}  
  
const mango = new User();  
console.log(mango); // {}
```

Конструктор класса

Метод `constructor` инициализирует экземпляр при создании. Вызывается автоматически при `new`, аргументы `new User(...)` становятся параметрами конструктора.

```
class User {  
  constructor(name, email) {  
    this.name = name;  
    this.email = email;  
  }  
}  
  
const mango = new User("Mango", "mango@mail.com");  
console.log(mango); // { name: 'Mango', email: 'mango@mail.com' }
```

`this` внутри конструктора указывает на новый создаваемый объект. Свойства `name`, `email` — **публичные свойства** (own-свойства экземпляра).

Объект параметров

Вместо длинного списка аргументов удобнее передавать один объект с именованными свойствами:

```
class User {  
  constructor(params) {  
    this.name = params.name;  
    this.email = params.email;  
  }  
}  
  
const mango = new User({ name: "Mango", email: "mango@mail.com" });
```

Методы класса

Функции, объявленные в классе после конструктора (без запятых между ними), доступны экземпляру через его прототип. Внутри используют `this`:


```
class User {
  constructor(params) {
    this.name = params.name;
    this.email = params.email;
  }

  getEmail() {
    return this.email;
  }

  changeEmail(newEmail) {
    this.email = newEmail;
  }
}

const mango = new User({ name: "Mango", email: "mango@mail.com" });
console.log(mango.getEmail()); // "mango@mail.com"
mango.changeEmail("new@mail.com");
console.log(mango.getEmail()); // "new@mail.com"
```

Прототип экземпляра

Методы класса не копируются в каждый экземпляр — незачем дублировать одну и ту же функцию тысячи раз. Вместо этого они хранятся в специальном объекте `User.prototype`, который автоматически становится прототипом (`[[Prototype]]`) каждого нового экземпляра:

```
console.log(User.prototype); // {constructor: f, getEmail: f, changeEmail: f}
```

Поэтому экземпляр может вызывать методы класса — они находятся у него в прототипе.

Приватные свойства

Инкапсуляция — принцип ООП, при котором детали внутренней реализации класса скрыты от внешнего кода; доступен только публичный интерфейс.

Символ `#` перед именем делает свойство приватным (доступ — только внутри класса). Приватное свойство обязательно нужно объявить заранее (до использования в конструкторе):

```
class User {
  name;
  #email;

  constructor(params) {
    this.name = params.name;
    this.#email = params.email;
  }

  getEmail() {
    return this.#email;
  }

  changeEmail(newEmail) {
    this.#email = newEmail;
  }
}

const mango = new User({ name: "Mango", email: "mango@mail.com" });
console.log(mango.getEmail()); // "mango@mail.com"
mango.changeEmail("mango@supermail.com");
console.log(mango.getEmail()); // "mango@supermail.com"
```

Прямое обращение `mango.#email` снаружи класса вызовет ошибку — доступ только через публичные методы.

Приватные методы

Так же через `#` можно скрыть и метод — например, внутреннюю проверку, которая не должна вызываться напрямую:

```
class User {
  name;
  #email;

  constructor(params) {
    this.name = params.name;
    this.#email = params.email;
  }

  getEmail() {
    return this.#email;
  }

  changeEmail(newEmail) {
    if (this.#validateEmail(newEmail)) {
      this.#email = newEmail;
    } else {
      console.log("Invalid email format");
    }
  }

  #validateEmail(email) {
    return email.includes("@");
  }
}

const mango = new User({ name: "Mango", email: "mango@mail.com" });
mango.changeEmail("newmail.com"); // "Invalid email format"
mango.changeEmail("new@mail.com");
console.log(mango.getEmail()); // "new@mail.com"

mango.#validateEmail("test"); // Ошибка — метод приватный
```

Геттеры и сеттеры

Геттер (`get`) и **сеттер** (`set`) — методы, которые снаружи выглядят как обычное публичное свойство, но позволяют выполнять код при чтении/записи. Их принято называть так же, как связанное свойство:

```

class User {
  #email;

  constructor(params) {
    this.name = params.name;
    this.#email = params.email;
  }

  get email() {
    return this.#email;
  }

  set email(newEmail) {
    this.#email = newEmail;
  }
}

const mango = new User({ name: "Mango", email: "mango@mail.com" });
console.log(mango.email); // "mango@mail.com" — вызвался геттер

mango.email = "mango@supermail.com"; // вызвался сеттер
console.log(mango.email); // "mango@supermail.com"

```

Внутри сеттера можно добавить проверку, как в обычном методе:

```

set email(newEmail) {
  if (newEmail === "") {
    console.log("Ошибка! Почта не может быть пустой строкой!");
    return;
  }
  this.#email = newEmail;
}

```

Геттеры/сеттеры удобны для простого чтения/записи (особенно приватных свойств), но не подходят для работы со свойствами-массивами или объектами.

Статические методы

Статические методы (static methods) — методы, доступные только самому классу, а не его экземплярам. Могут быть публичными и приватными.

```

class MyClass {
  static myMethod() {
    console.log("A static method");
  }
}

MyClass.myMethod(); // "A static method"

```

Пример: статическое свойство `#takenEmails` хранит занятые почты, статический метод `isEmailTaken` проверяет доступность почты. При создании экземпляра почта добавляется в общий список:

```

class User {
  static #takenEmails = [];

  static isEmailTaken(email) {
    return User.#takenEmails.includes(email);
  }

  #email;

  constructor(params) {
    this.#email = params.email;
    User.#takenEmails.push(params.email);
  }
}

const mango = new User({ email: "mango@mail.com" });

console.log(User.isEmailTaken("poly@mail.com")); // false
console.log(User.isEmailTaken("mango@mail.com")); // true

```

Внутри статического метода `this` ссылается на сам класс, а не на экземпляр. Поэтому статический метод имеет доступ к статическим свойствам класса, но не к свойствам экземпляра — логично, ведь его вызывает сам класс.

Наследование классов

Ключевое слово `extends` реализует наследование: дочерний (производный) класс наследует свойства и методы родительского класса.

```

class Parent {}

class Child extends Parent {
  // ...
}

```

Смысл: базовый класс хранит общие свойства и методы для группы производных классов (например, роли пользователей — админ, копирайтер, контент-менеджер имеют общее: почту, пароль, но и уникальные особенности). Если создавать независимые классы для каждой роли — общий код дублируется, и изменения приходится вносить везде. Решение — вынести общее в базовый класс `User`, а роли наследовать от него.

```

class User {
  #email;

  constructor(email) {
    this.#email = email;
  }

  get email() {
    return this.#email;
  }

  set email(newEmail) {
    this.#email = newEmail;
  }
}

class ContentEditor extends User {
  // Тело класса ContentEditor
}

const editor = new ContentEditor("mango@mail.com");
console.log(editor.email); // "mango@mail.com"

```

ContentEditor наследует от User конструктор, геттер/сеттер email и приватное свойство #email.

Конструктор дочернего класса

В конструкторе дочернего класса обязательно нужно вызвать `super(args)` — это вызов конструктора родительского класса. Без этого обращение к `this` в конструкторе дочернего класса вызовет ошибку.

```

class User {
  #email;

  constructor(email) {
    this.#email = email;
  }

  get email() { return this.#email; }
  set email(newEmail) { this.#email = newEmail; }
}

class ContentEditor extends User {
  constructor(params) {
    // Вызов конструктора родительского класса User
    super(params.email);

    this.posts = params.posts;
  }
}

const editor = new ContentEditor({ email: "mango@mail.com", posts: [] });
console.log(editor.email); // "mango@mail.com"

```

Методы дочернего класса

Дочерний класс использует методы и свойства родителя, а также может объявлять собственные методы, доступные только его экземплярам:

```
// Выше объявлен класс User

class ContentEditor extends User {
  constructor(params) {
    super(params.email);
    this.posts = params.posts;
  }

  addPost(post) {
    this.posts.push(post);
  }
}

const editor = new ContentEditor({ email: "mango@mail.com", posts: [] });

editor.addPost("post-1");
editor.addPost("post-2");
console.log(editor.posts); // ['post-1', 'post-2']
```

`addPost` — метод дочернего класса `ContentEditor`, доступен только его экземплярам (у `User` его нет).